

Les tests

Pourquoi on test

- Éviter les régressions
- Harnais de sécurité lors de remaniements
- Documentation. Exemple d'utilisation
- Vérifie l'exactitude de ce qui est produit vs ce qui est demandé

Types de tests

- Tests unitaires (Écrit par le développeur)
- Tests d'acceptation (Acceptance tests) (Tests qui vérifie la logique d'affaire)
- Tests d'intégration (Tests de plomberie, vérifie qu'un module fonctionne adéquatement)
- Tests systèmes (Tests d'intégration au niveau de l'ensemble des modules)

Autres types de tests

- Test de performance (Load test, stress test, capacity test)
- Test d'intrusion (sécurité)
- Test utilisateur (test d'utilisabilité)

Composantes d'un test

- Les 3 'A' (en anglais)
 - Arrange
 - Prépare la classe que l'on veut tester
 - Act
 - Appel de la fonctionnalité à tester
 - Assert
 - Vérifie que l'appel à fonctionné adéquatement

JUnit

- Éléments de JUnit (À faire dans IntelliJ avec JUnit 5)
 - @Test, @BeforeEach, @AfterEach, @BeforeAll, @AfterAll
 - Exception (assertThrows)
 - @ExtendWith(MockitoExtension.class)
 - AssertJ
 - <http://joel-costigliola.github.io/assertj/>

TDD

- 3 lois du TDD
 - Pas le droit d'écrire du code de production avant d'avoir un test qui échoue. Ne pas compiler est équivalent à un test qui échoue
 - Écrire le code de production afin de satisfaire le test et uniquement le test (évite YAGNI)
 - Remanier le code de production ET les tests afin de garder les 2 propres. Les tests sont des citoyens de premières classes tout comme l'est le code de production

TDD

- Avantages
 - Permet de réduire le nombre de régression de façon substantielle
 - On n'écrit seulement ce qu'on a besoin (YAGNI)
 - Oblige d'écrire du code qui est facile à tester
 - Rend le code plus robuste puisqu'il ne dépend que de méthodes strictement nécessaires
 - Permet de réduire l'effort au débogage
 - Fournit des exemples de documentation (rappelez-vous des commentaires dans le code)

Quoi tester

- Préférer l'interface publique d'une classe
 - Permet d'obtenir des tests plus robustes (c'est quoi robuste déjà?)
 - Moins sujet au changement
- S'assurer de ne tester qu'une seule fois une méthode (1 fois pour happy day, 1 fois pour exception)

Exercice

- Faire une 'Pile' (Stack) en TDD avec la classe